

Numerical Integration Tool

C++ Programming Project



Gaël KERNINON

Cracow University of Technology, Poland 

May 26, 2026

Contents

1	Introduction	2
2	Project Architecture	2
3	Getting Started	3
3.1	Prerequisites	3
3.2	Compilation	3
3.3	Execution	3
4	Usage Guide	3
5	Mathematics	4
5.1	Integration Methods	4
5.2	Implementation	4
5.3	Adaptive Precision	5
6	Execution Results	5

1 Introduction

This project provides a C++ implementation of numerical integration using the **Trapezoidal Rule** and **Simpson's Rule**. It provides an adaptive algorithm that iteratively doubles the number of intervals until a user-given precision target is achieved. It is designed to evaluate both standard test functions and custom mathematical functions.

2 Project Architecture

The project is divided into three main files to maintain modularity :

```

1 Project
2 |--- givenfunction.cpp
3 |--- integration.cpp
4 |--- main.cpp
5 |--- report.pdf
6 |--- README.md

```

- **givenfunction.cpp**: Contains the mathematical function to be integrated (By default, $f(x) = \cos(x) + x^3$).
- **integration.cpp**: Contains the main numerical algorithms (Trapezoidal Rule, Simpson's Rule, and the integration loop).
- **main.cpp**: The main code that handles user input, prints the tests, calculates analytical errors, and prints the formatted results.

3 Getting Started

3.1 Prerequisites

To compile and run this project, you need a standard C++ compiler¹ installed on your system.

3.2 Compilation

Open your terminal or command prompt, navigate to the directory containing the project files, and compile `main.cpp` with your compiler. Because `main.cpp` includes the other `.cpp` files directly via `#include`, you only need to compile the main file:

```
1 c++ main.cpp -o numerical_integration
```

3.3 Execution

Run the compiled executable from the terminal:

Linux/MacOS:

```
1 ./numerical_integration
```

Windows 11 (Powershell):

```
1 .\numerical_integration.exe
```

4 Usage Guide

While running the program, you will be prompted to enter three parameters:

1. **a:** The lower bound of the integration interval.
2. **b:** The upper bound of the integration interval.
3. **precision:** The desired accuracy (e.g., `1e-6` for six decimal places of precision).

Example Interaction:

```
1 ===== Numerical Integration Tester =====
2 Enter integration interval [a,b]:
3 a = 0
4 b = 3.14159
5 Enter desired precision (e.g., 1e-6): 1e-6
6 =====
```

The program will then automatically run Test 1 (x^2), Test 2 ($\sin(x)$), and finally the Custom Function, displaying the numerical result, the number of intervals (n) required, and the error compared to the exact analytical solution for both Tests 1 and 2.

¹e.g., GCC, Clang, or MSVC

5 Mathematics

5.1 Integration Methods

- **Trapezoidal Rule:** Approximates the area under the curve using trapezoids.

$$\int_a^b f(x) dx \approx \frac{h}{2} \left[f(a) + 2 \sum_{i=1}^{n-1} f(x_i) + f(b) \right] \quad (1)$$

- **Simpson's Rule:** Uses piecewise quadratic parabolas for a smoother, faster-converging approximation (requires an even number of intervals).

$$\int_a^b f(x) dx \approx \frac{h}{3} \left[f(a) + 4 \sum_{\text{odd } i} f(x_i) + 2 \sum_{\text{even } i} f(x_i) + f(b) \right] \quad (2)$$

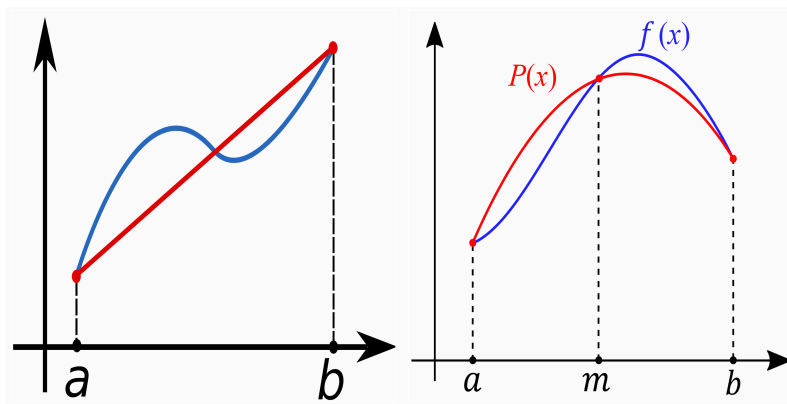


Figure 1: Trapezoidal rule (left) and Simpson's Rule (right) approximating functions (in blue)

5.2 Implementation

Trapezoidal Rule

```

1 double trapezoidalRule(function<double(double)> f, double a, double b, int
   n)
2 {
3     // Approximates the integral of f(x) from a to b using n intervals.
4     double h = (b - a) / n;
5     double sum = 0.5 * (f(a) + f(b));
6     for (int i = 1 ; i < n ; i++)
7         sum += f(a + i * h);
8     return sum * h;
9 }

```

Simpson's Rule

```

1 double simpsonsRule(function<double(double)> f, double a, double b, int n)
2 {
3     if (n % 2 == 1)
4         n++; // number of intervals must be even
5
6     double h = (b - a) / n;
7     double sum = f(a) + f(b);
8
9     for (int i = 1 ; i < n ; i++)
10    {
11        double x = a + i * h;
12        if (i % 2 == 0)
13            sum += 2 * f(x); // Even
14        else
15            sum += 4 * f(x); // Odd
16    }
17    return sum * h / 3.0;
18 }

```

5.3 Adaptive Precision

Instead of guessing the number of intervals (n), the program uses a **while** loop starting at $n = 2$. It calculates the integral, then doubles n and calculates it again. If the absolute difference between the new result and the previous result is greater than the requested precision, the loop continues. A maximum of $n = 1,000,000$ prevents infinite loops.

6 Execution Results

```

1 ===== Test 1 =====
2 Integrating f(x) = x^2 on [1,5]
3 Exact Analytical Solution : 41.3333
4
5 Trapezoidal Rule Result   : (n=8192)
6     Value = 41.3333 | Error = 1.58946e-07
7
8 Simpson's Rule Result     : (n=4)
9     Value = 41.3333 | Error = 7.10543e-15
10 =====
11
12 ===== Test 2 =====
13 Integrating f(x) = sin(x) on [1,5]
14 Exact Analytical Solution : 0.25664
15
16 Trapezoidal Rule Result   : (n=1024)
17     Value = 0.25664 | Error = 3.26335e-07
18
19 Simpson's Rule Result     : (n=64)
20     Value = 0.25664 | Error = 2.17658e-08
21 =====
22
23
24 ===== Given Function =====
25 Integrating given function f(x) on [1,5]
26 Trapezoidal Rule Result   : (n=32768)
27     Value = 154.2
28
29 Simpson's Rule Result     : (n=128)
30     Value = 154.2
31 =====

```